



CS231. Nhập môn Thị giác máy tính

Đặc trưng màu sắc



Mục tiêu

- Sinh viên hiểu về **màu sắc trong ảnh**
- Áp dụng các phương pháp **trích xuất đặc trưng dựa trên màu sắc** để phân tích và nhận diện đối tượng trong ảnh.

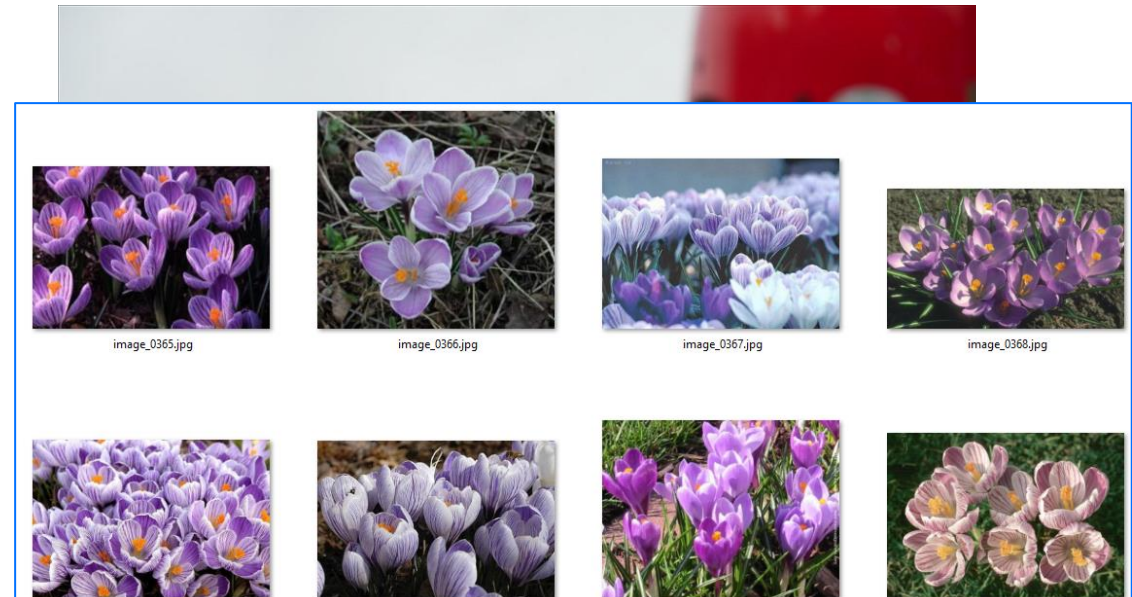
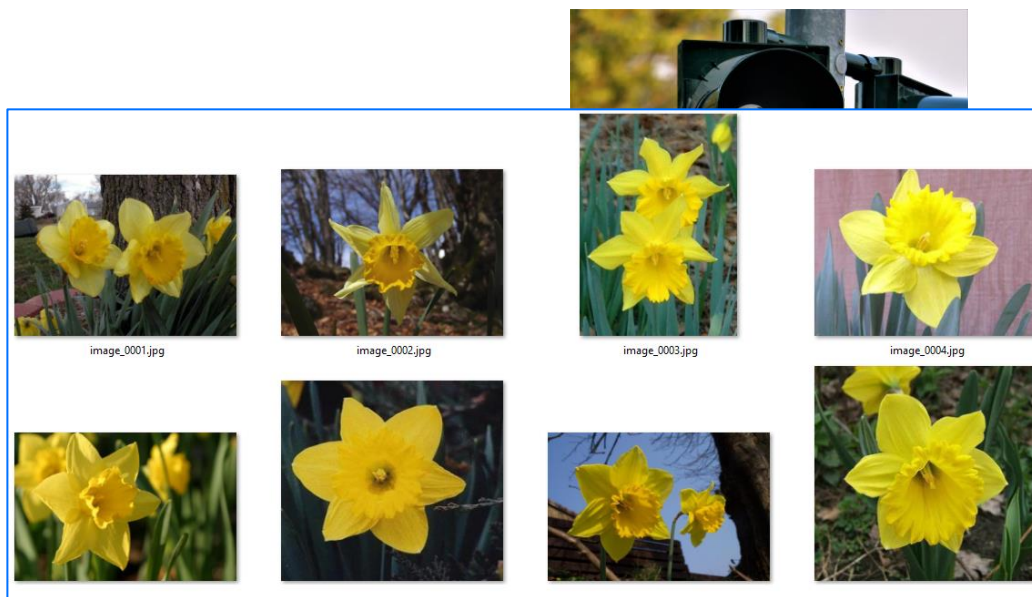
Dẫn nhập

- Màu sắc đóng vai trò quan trọng trong cuộc sống.



Dẫn nhập

- Màu sắc đóng vai trò quan trọng trong cuộc sống.
- Có thể phân biệt các đối tượng, vật thể khác nhau dựa trên màu sắc





Các đặc trưng màu sắc phổ biến

- Histogram màu
- Đặc trưng moment màu
- Màu sắc chủ đạo của ảnh
- Color Correlogram



Histogram màu

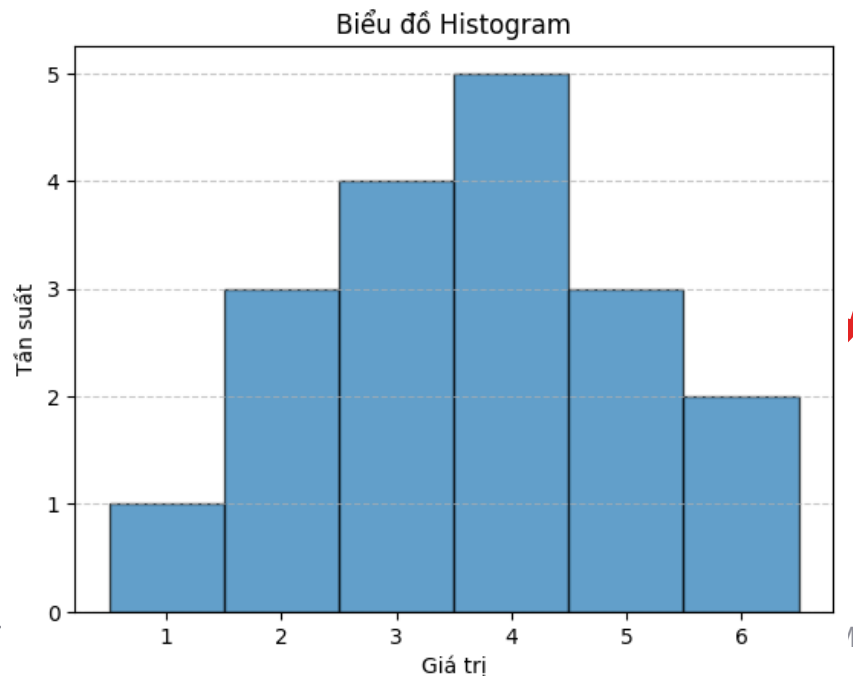
Color Histogram

Định nghĩa Histogram

data = [1, 3, 2, 2, 4, 6, 3, 3, 5, 4, 2, 4, 5, 4, 5, 4, 3, 6]

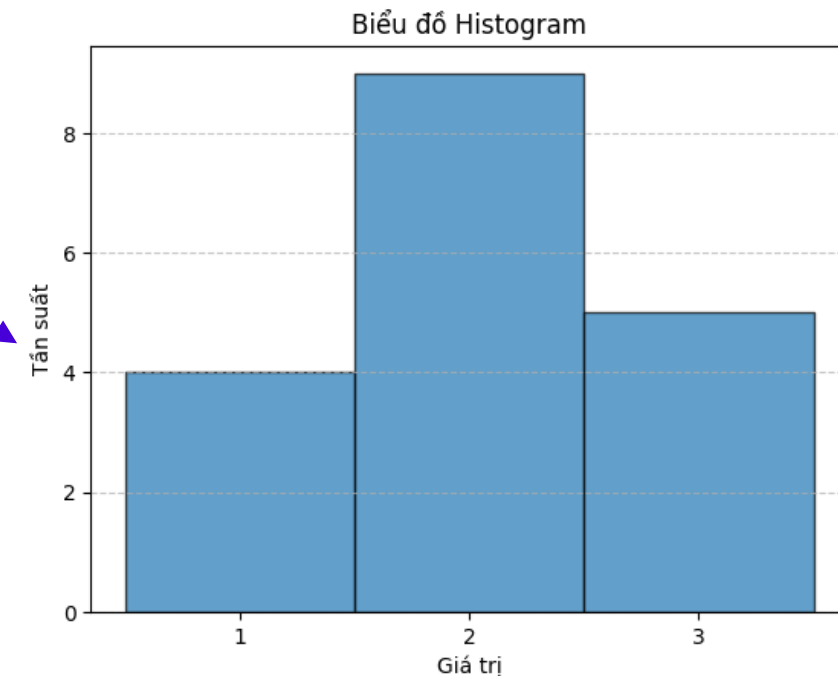


Màu	1	2	3	4	5	6
Số lần xuất hiện	1	3	4	5	3	2



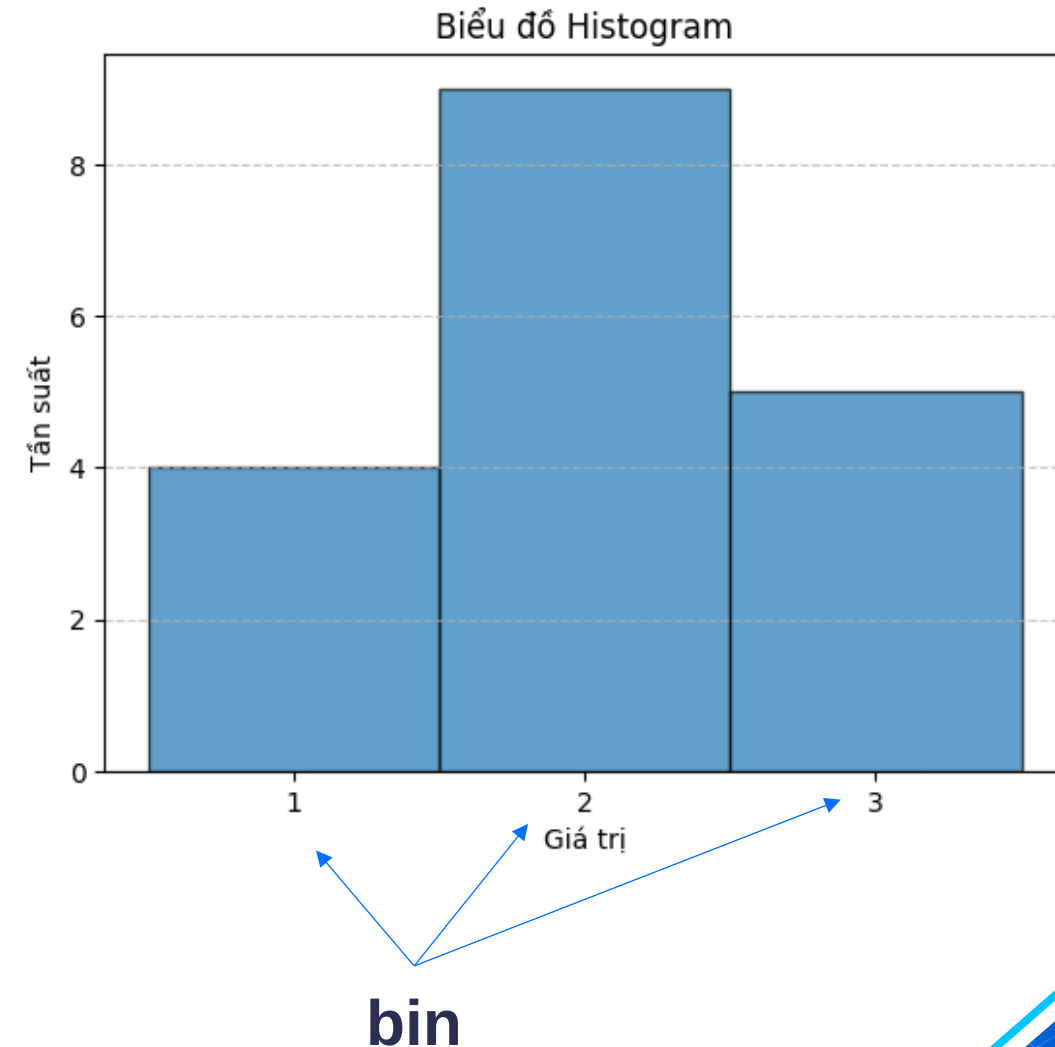
6 bin

3 bin:
bin 1 = 1-2
bin 2 = 3-4
bin 3 = 5-6



Định nghĩa Histogram

- **Histogram** là một dạng biểu đồ cột thể hiện tần suất xuất hiện (số lượng) của các giá trị.
- **Mỗi cột** trong biểu đồ đại diện cho một **(khoảng) giá trị**, được gọi là **bin**.





Histogram màu trong ảnh

- **Histogram màu** thể hiện **tần suất xuất hiện** của **từng mức màu** trong một không gian màu.
 - Gray scale
 - RGB (Red, Green, Blue)
 - HSV (Hue, Saturation, Value)
 - Lab (Luminance, a, b)



Cách tính Histogram màu

1. Xác định không gian màu của ảnh

- Gray scale $\rightarrow$ có 1 kênh màu $\rightarrow$ miền giá trị $[0 - 255]$
- RGB $\rightarrow$ có 3 kênh màu $\rightarrow [0-255, 0-255, 0-255] = 16.777.216$
- HSV $\rightarrow$ có 3 kênh màu $\rightarrow [0-179, 0-255, 0-255]$
- Lab $\rightarrow$ có 3 kênh màu $\rightarrow [0-255, 0-255, 0-255]$

Cách tính Histogram màu

2. Chia giá trị mỗi kênh thành các khoảng giá trị (bins)

- Mục đích để giảm số lượng giá trị cần đếm
- Ví dụ: Gray scale $\rightarrow$ miền giá trị $[0 - 255]$

Chọn số bin = 256

bin	Ý nghĩa
0	số lượng pixel có giá trị 0
1	số lượng pixel có giá trị 1
...	...
255	số lượng pixel có giá trị 255

Chọn số bin = 128

bin	Ý nghĩa
0	số lượng pixel có giá trị 0, 1
1	số lượng pixel có giá trị 2, 3
...	...
127	số lượng pixel có giá trị 254, 255

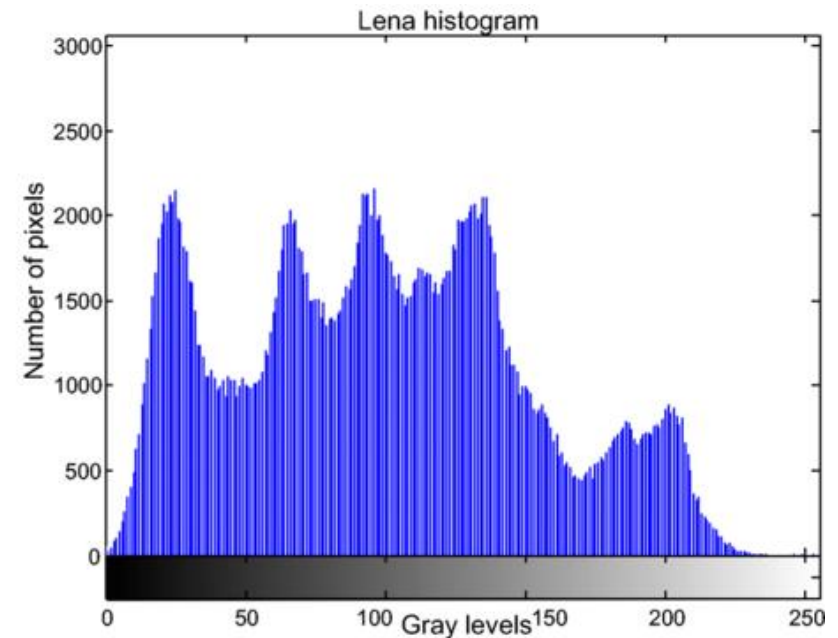
Cách tính Histogram màu

3. Đếm số lượng pixel trong mỗi bin

Ví dụ: Gray scale



(a)

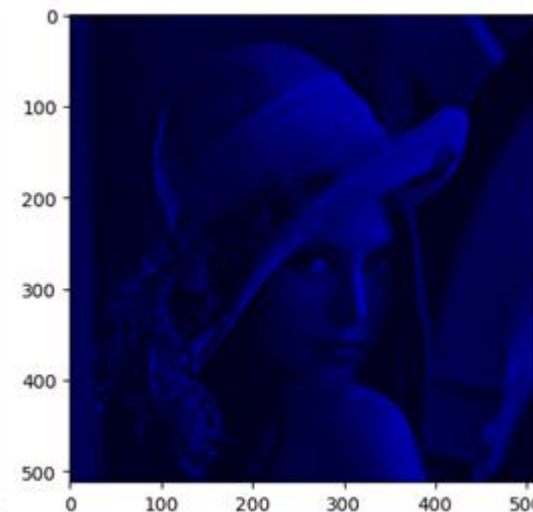
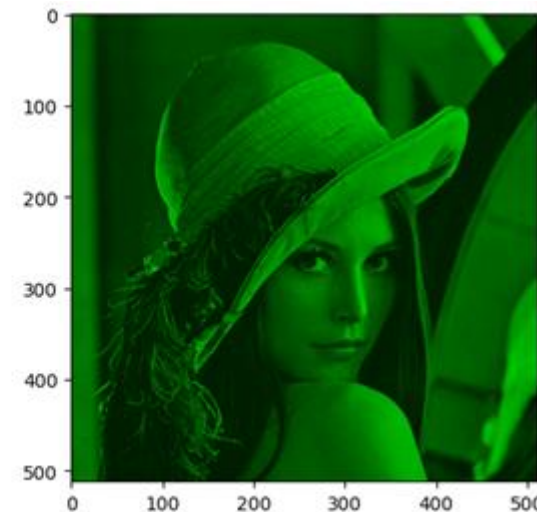
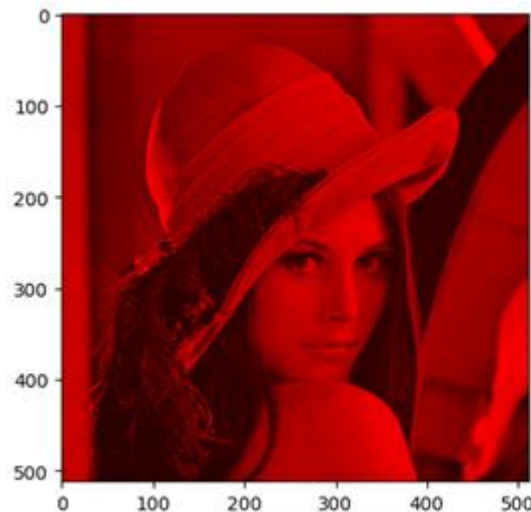


(b)

https://www.researchgate.net/figure/a-Original-lena-image-b-Histogram-of-lena-image_fig1_335591569

Cách tính Histogram màu

Ví dụ: RGB $\rightarrow$ có 3 kênh màu $\rightarrow [0-255, 0-255, 0-255] =$
16.777.216 màu !!!





Code hiển thị Histogram màu

```
import cv2
import matplotlib.pyplot as plt

def display_histogram_RGB(image_path, bins=256):
    # Đọc ảnh
    image = cv2.imread(image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Chuyển từ BGR sang RGB

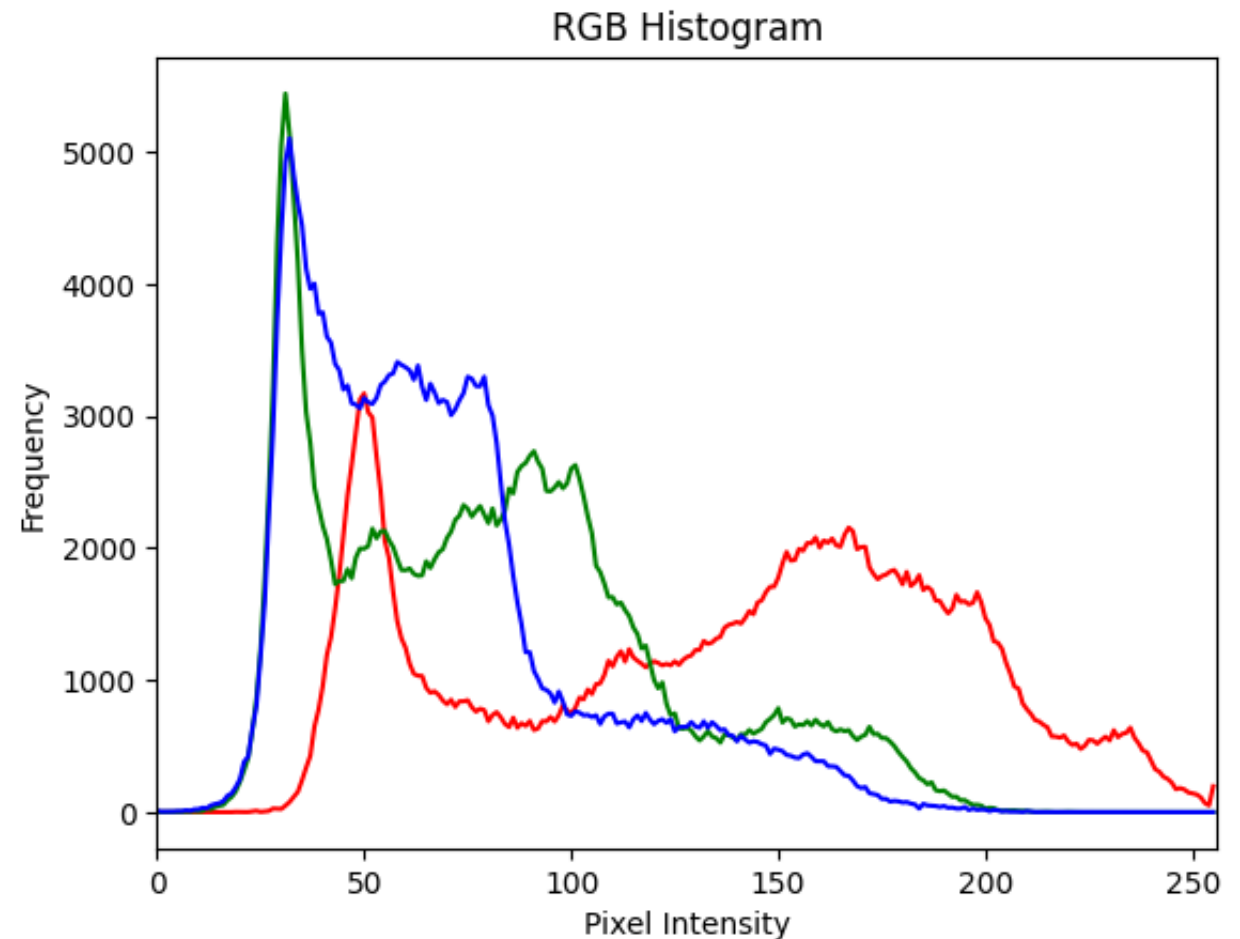
    # Tạo histogram cho từng kênh màu
    colors = ('r', 'g', 'b')
    plt.figure()
    plt.title('RGB Histogram')
    plt.xlabel('Pixel Intensity')
    plt.ylabel('Frequency')

    for i, color in enumerate(colors):
        hist = cv2.calcHist([image], [i], None, [bins], [0, 256])
        plt.plot(hist, color=color)

    plt.xlim([0, 256])
    plt.show()
```

Code hiển thị Histogram màu

```
# Thay thế 'image_path' bằng đường dẫn đến ảnh của bạn  
image_path = "/content/lena.jpg"  
display_histogram_RGB(image_path)
```



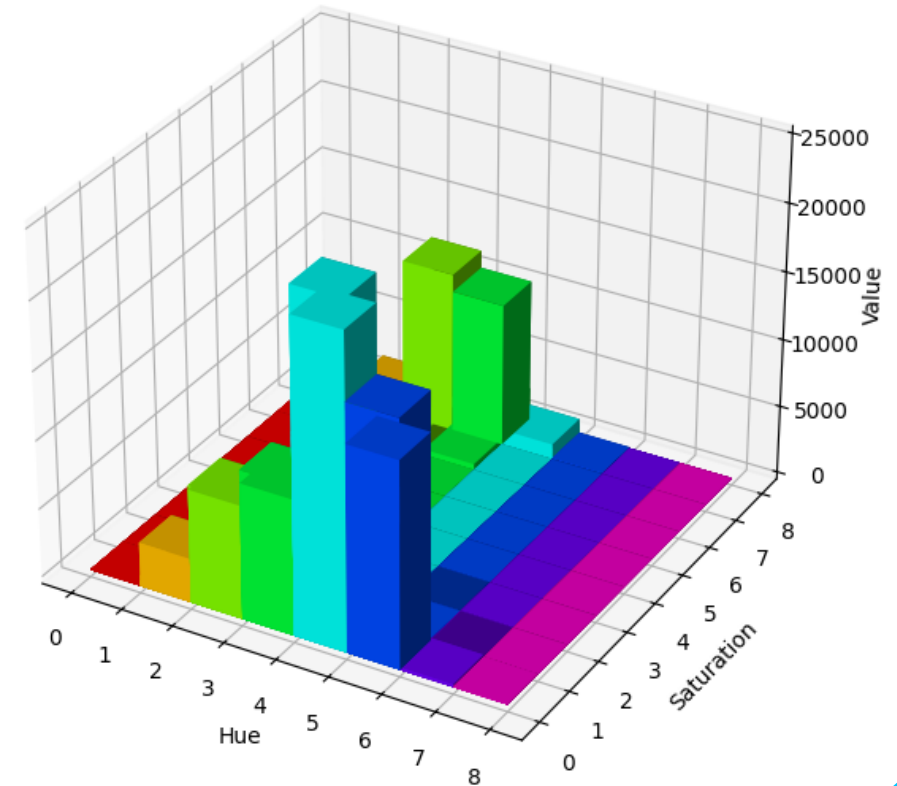
Cách tính Histogram màu

Ví dụ: HSV $\rightarrow$ có 3 kênh màu $\rightarrow [0-179, 0-255, 0-255]$

3D HSV Histogram



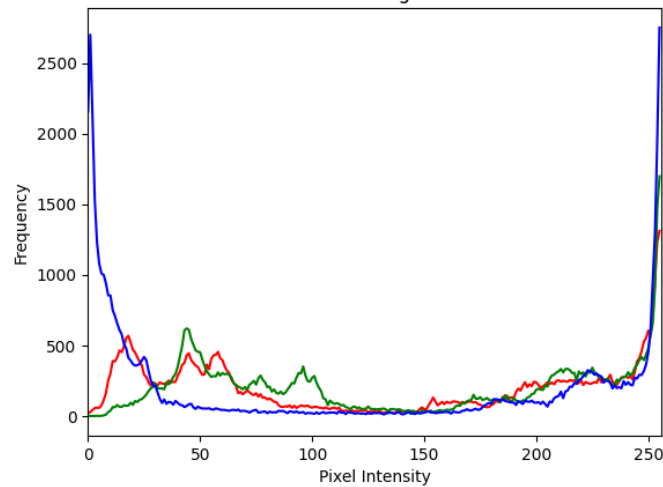
Chọn số bin = (8,8,8)



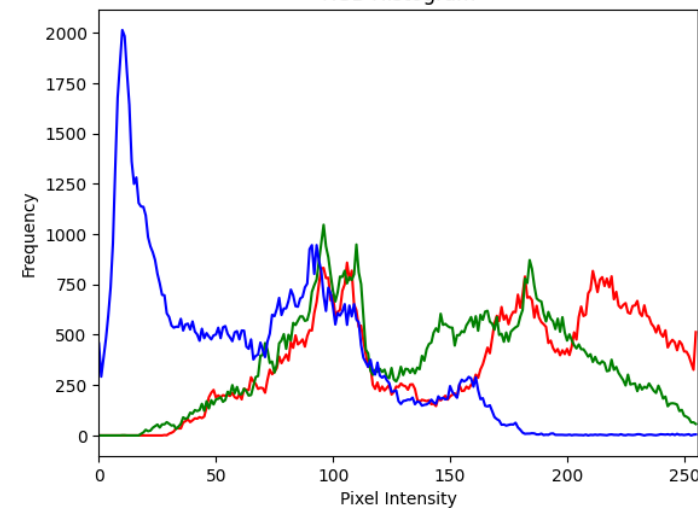
Sử dụng Histogram làm vector đặc trưng



RGB Histogram



RGB Histogram





Sử dụng Histogram làm vector đặc trưng

```
"""Trích xuất histogram màu trong không gian RGB."""  
def extract_color_histogram_rgb(image_path, bins=256):  
  
    image = cv2.imread(image_path)    # Đọc ảnh  
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Chuyển từ BGR sang RGB  
  
    # Tính histogram và chuẩn hóa thành feature vector  
    feature_vector = []  
    for i in range(3): # 3 kênh màu R, G, B  
        hist = cv2.calcHist([image], [i], None, [bins], [0, 256])  
        hist = cv2.normalize(hist, hist).flatten() # Chuẩn hóa và làm phẳng  
        feature_vector.extend(hist)  
  
    # Chuẩn hóa và làm phẳng  
    feature_vector = np.array(feature_vector)  
    feature_vector = cv2.normalize(feature_vector, feature_vector).flatten()  
  
    return feature_vector # Kích thước 256+256+256 = 768
```



Sử dụng Histogram làm vector đặc trưng

```
def extract_color_histogram(image, bins=(8, 8, 8)):  
    """Trích xuất histogram màu trong không gian HSV."""  
    image = cv2.imread(image_path) # Đọc ảnh  
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)  
    hist = cv2.calcHist([hsv], [0, 1, 2], None, bins, [0, 180, 0, 256, 0, 256])  
    cv2.normalize(hist, hist)  
    return hist.flatten()
```




Histogram màu

- **Ưu điểm:** Đơn giản, dễ tính toán
- **Nhược điểm:** không biểu thị được thông tin không gian (vị trí) của màu sắc trong ảnh.





Đặc trưng moment màu

Color Moments

https://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/AV0405/KEEN/av_as2_nkeen.pdf

Moment màu

- Là một phương pháp mô tả phân phối màu sắc bằng cách sử dụng các giá trị thống kê:
 - Trung bình (mean),
 - Độ lệch chuẩn (standard deviation),
 - Độ nghiêng (skewness).

Ý nghĩa của từng đặc trưng:

- **Trung bình (mean):** Thể hiện màu trung bình trong ảnh.
- Giá trị trung bình của kênh màu thứ i trong ảnh được tính như sau:

$$E_i = \sum_N^{j=1} \frac{1}{N} p_{ij}$$

p_{ij} là giá trị màu của pixel thứ j trong kênh màu i .
 N là tổng số pixel

Ý nghĩa của từng đặc trưng:

- **Độ lệch chuẩn (standard deviation):** Độ lan truyền của các giá trị màu xung quanh giá trị trung bình.

$$\sigma_i = \sqrt{\frac{1}{N} \sum_{j=1}^N (p_{ij} - E_i)^2}$$

p_{ij} là giá trị màu của pixel thứ j trong kênh màu i .
 N là tổng số pixel

Ý nghĩa của từng đặc trưng:

- **Độ nghiêng (skewness):** Độ bất đối xứng của phân phối màu

$$s_i = \sqrt[3]{\frac{1}{N} \sum_{j=1}^N (p_{ij} - E_i)^3}$$

p_{ij} là giá trị màu của pixel thứ j trong kênh màu i .
 N là tổng số pixel



Rút trích đặc trưng Moment màu

```
def extract_color_moments(image_path):  
    """Tính toán moment màu (mean, variance, skewness) trên kênh HSV."""  
    image = cv2.imread(image_path) # Đọc ảnh  
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)  
    mean = np.mean(hsv, axis=(0, 1))  
    std = np.std(hsv, axis=(0, 1))  
    skewness = np.mean(((hsv - mean) / (std + 1e-6)) ** 3, axis=(0, 1))  
    return np.concatenate([mean, std, skewness])
```



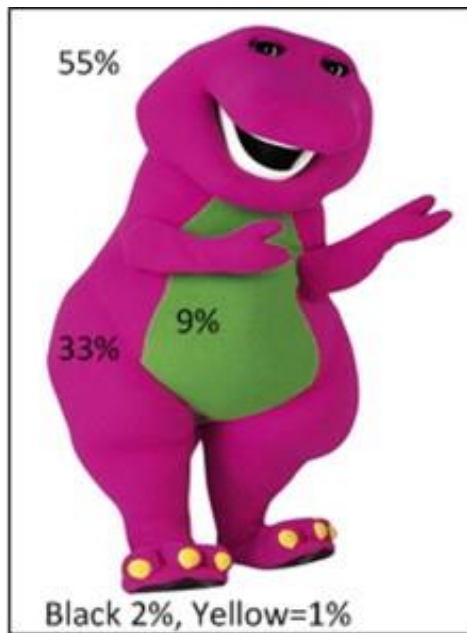


Màu sắc chủ đạo của ảnh

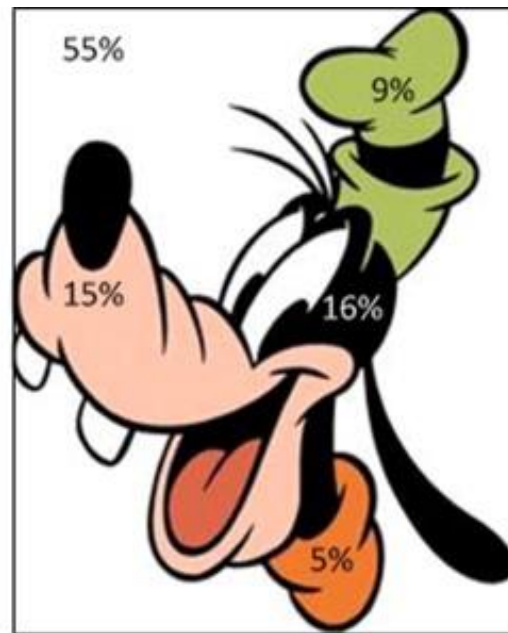
Dominant Color Descriptor (DCD)

Màu sắc chủ đạo của ảnh

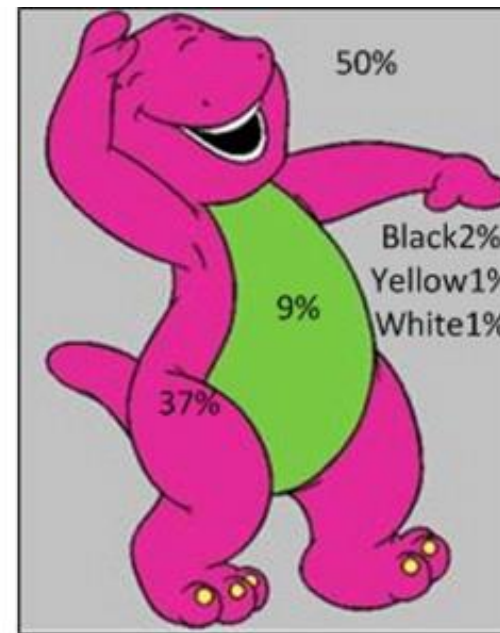
- Mô tả các **màu sắc nổi bật nhất** (chủ đạo)



(A) Image 1



(B) Image 2

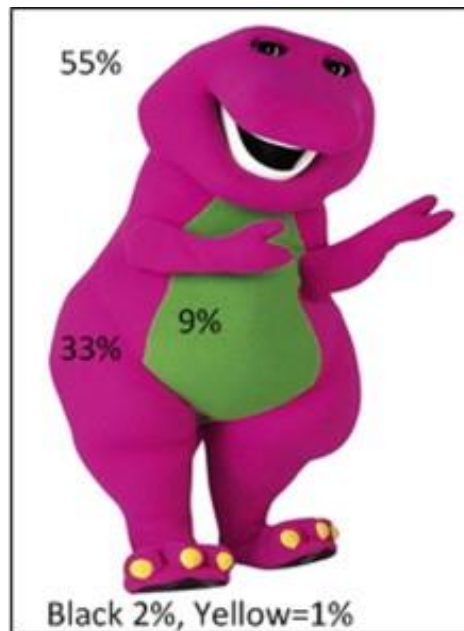


(C) Image 3

[A weighted dominant color descriptor for content-based image retrieval - ScienceDirect](#)

Màu sắc chủ đạo của ảnh

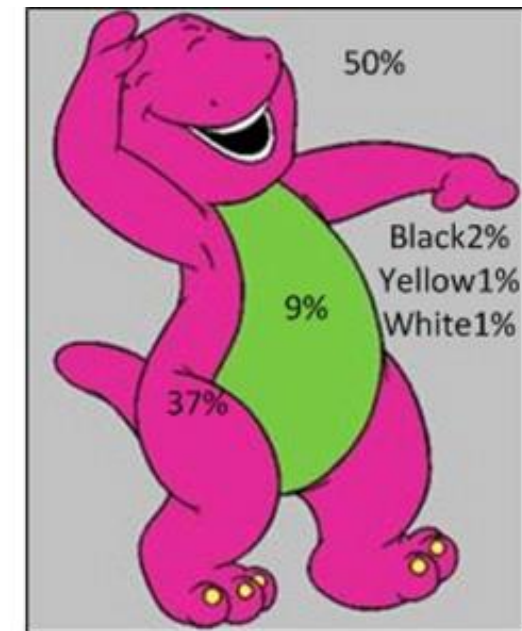
- Phương pháp xác định: Chia nhỏ các vùng **màu chủ đạo** và xác định **tỷ lệ xuất hiện** của chúng.



(A) Image 1

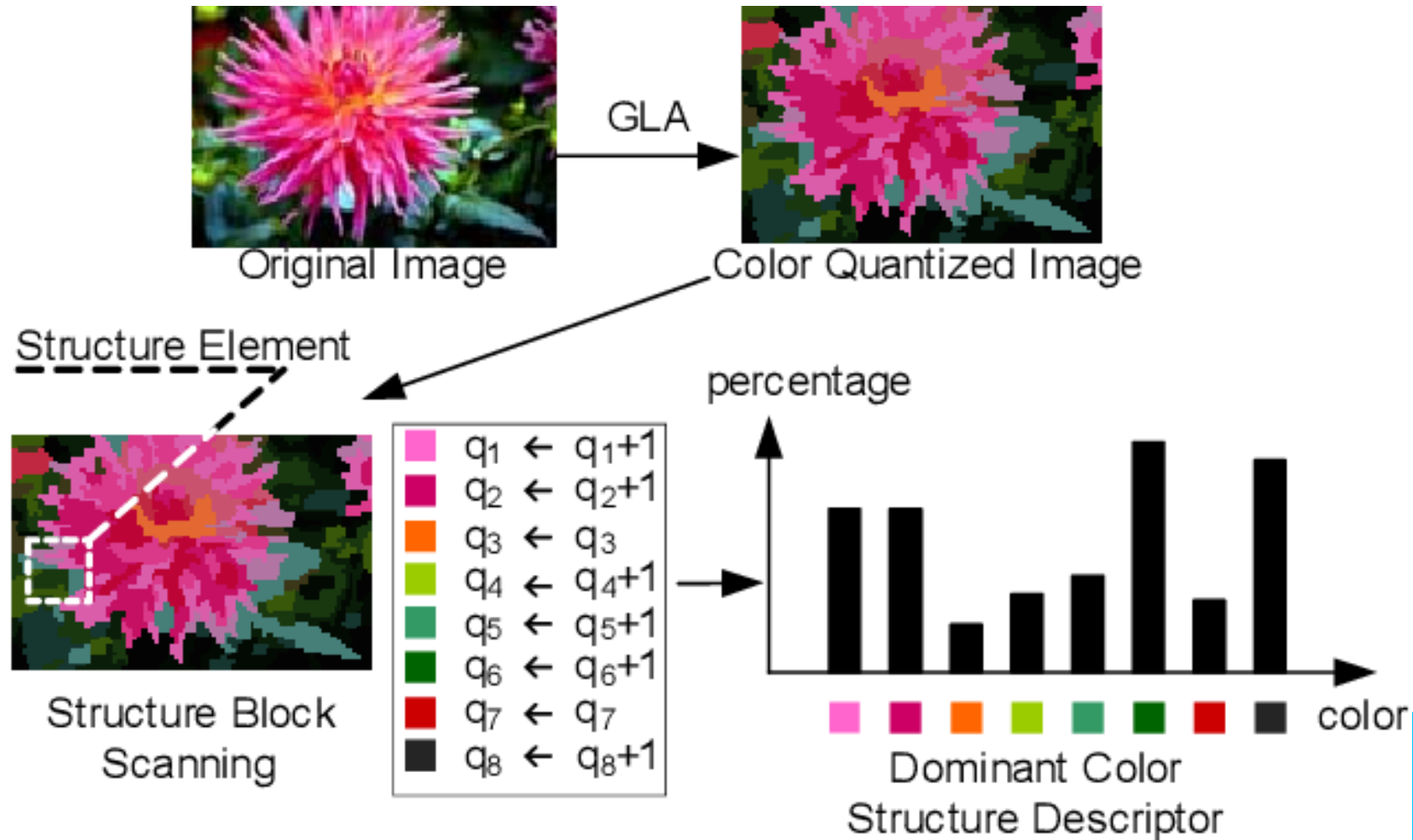


(B) Image 2



(C) Image 3

Màu sắc chủ đạo của ảnh



[\[PDF\] Dominant Color Structure Descriptor for Image Retrieval | Semantic Scholar](#)

Fig. 1 Dominant Color Structure Descriptor extraction

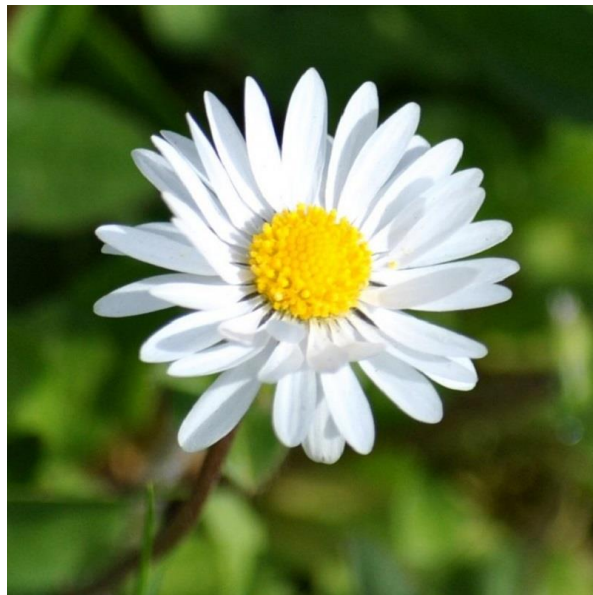


Code trích xuất đặc trưng

```
def extract_dominant_color(image_path, k=3):  
    """Trích xuất màu sắc chủ đạo bằng K-means clustering."""  
    image = cv2.imread(image_path) # Đọc ảnh  
    pixels = image.reshape(-1, 3)  
    kmeans = cv2.KMEANS_RANDOM_CENTERS  
    _, labels, centers = cv2.kmeans(np.float32(pixels), k, None,  
                                    (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0),  
                                    10, kmeans)  
    hist = np.bincount(labels.flatten(), minlength=k) / labels.size  
    return np.hstack([centers.flatten(), hist])
```

Lưu ý: Độ đo khoảng cách

- Vì mỗi ảnh có thể có **số lượng màu chủ đạo khác nhau** hoặc **vị trí các màu khác nhau**, ta cần một cách đo khoảng cách phù hợp.



Lưu ý: Độ đo khoảng cách

(a) Khoảng cách Euclidean trung bình

- Với hai ảnh có tập hợp **k màu chủ đạo**, ta tính khoảng cách Euclidean giữa từng cặp màu gần nhất.
- Sau đó, lấy **trung bình khoảng cách của các cặp gần nhất**.

$$D(A, B) = \frac{1}{k} \sum_{i=1}^k \min_j ||C_A^i - C_B^j||$$



Lưu ý: Độ đo khoảng cách

(b) Khoảng cách Earth Mover's Distance (EMD)

- Phương pháp này xem tập hợp màu chủ đạo của mỗi ảnh như một phân phối xác suất và tính toán chi phí tối thiểu để chuyển đổi một phân phối này sang phân phối kia.
- Phù hợp khi số lượng màu chủ đạo trong hai ảnh không bằng nhau.
- OpenCV hỗ trợ tính toán bằng `cv2.EMD()`.





Color Coherence Vector (CCV)

<https://www.cs.cornell.edu/~rdz/Papers/PZM-MM96.pdf>

Color Coherence Vector (CCV)

- Là một phương pháp mở rộng của **histogram màu** giúp phân biệt giữa các vùng màu liên kết (coherent) và không liên kết (incoherent) trong ảnh.
- Biểu diễn mối quan hệ giữa các màu sắc
- CCV giúp phân biệt được các đối tượng có màu sắc giống nhau nhưng có phân bố khác nhau trong không gian ảnh.

1. Ý tưởng chính

- **Histogram màu** chỉ lưu trữ tần suất xuất hiện của từng màu, nhưng không quan tâm đến sự phân bố không gian của màu sắc trong ảnh.



Figure 1: Two images with similar color histograms

<https://www.cs.cornell.edu/~rdz/Papers/PZM-MM96.pdf>

1. Ý tưởng chính

- **CCV** khắc phục điều này bằng cách chia mỗi màu thành **hai nhóm**:
 - **Coherent pixels**: Các điểm ảnh thuộc vùng màu lớn, có kết nối với nhau.
 - **Incoherent pixels**: Các điểm ảnh thuộc vùng nhỏ, không liên kết đáng kể.
- Nhờ đó, CCV giúp **phân biệt rõ hơn giữa ảnh có cấu trúc và ảnh bị nhiễu**



2. Các bước tính CCV

1. Chuyển đổi không gian màu (ví dụ: từ RGB sang HSV hoặc Lab).
- 2. Lượng tử hóa màu:** để giảm số lượng màu còn n màu phân biệt (ví dụ: 64 hoặc 256 mức).
3. Phân vùng ảnh bằng thuật toán **xác định các thành phần liên thông** (connected component analysis).

2. Các bước tính CCV

4. Gán nhãn cho các điểm ảnh :

- Nếu một vùng màu có **kích thước lớn hơn ngưỡng $\tau \rightarrow$ nhóm coherent.**
- Nếu **nhỏ hơn $\tau \rightarrow$ nhóm incoherent.**

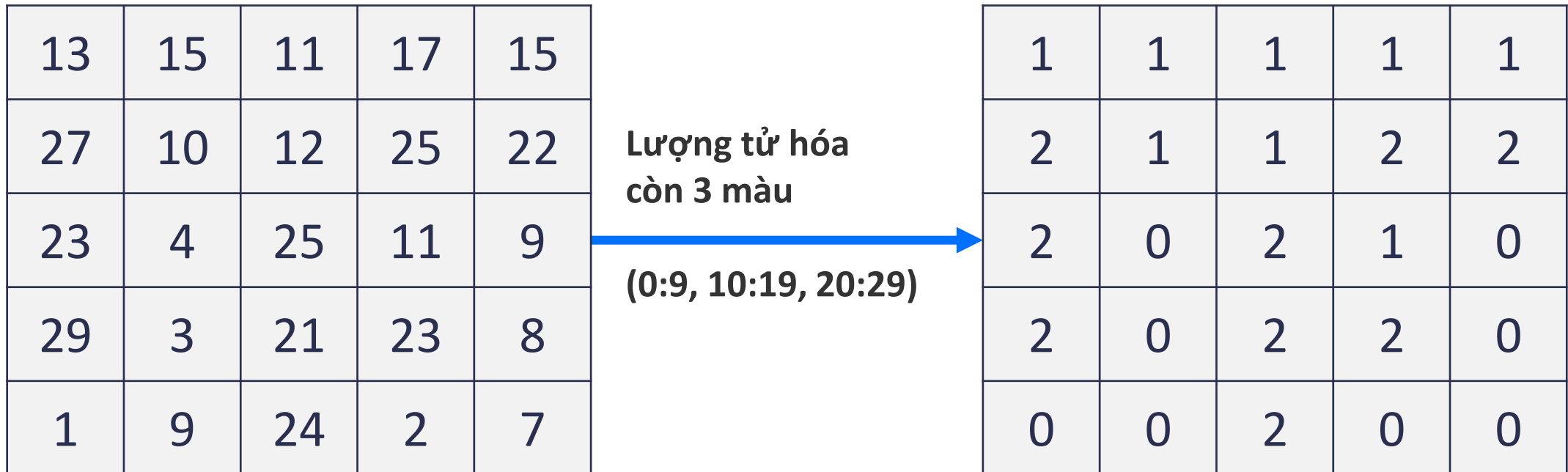
(Giá trị tham số τ do người dung xác định, thường là 1% kích thước ảnh)

5. Tạo vector CCV: Với mỗi màu c , lưu số lượng pixel của nhóm coherent (α_c) và incoherent (β_c):

$$CCV = \{(\alpha_1, \beta_1), (\alpha_2, \beta_2), \dots, (\alpha_n, \beta_n)\}$$

2. Các bước tính CCV

- Ví dụ:



[Image Retrieval: Color Coherence Vector - Owlcation](#)

2. Các bước tính CCV

- Ví dụ:



Image Retrieval: Color Coherence Vector - Owlcation

2. Các bước tính CCV

• Ví dụ:

1	1	1	1	1
2	1	1	2	2
2	0	2	1	0
2	0	2	2	0
0	0	2	0	0

Phân loại với
ngưỡng $\tau = 4$

Màu 0, có 2 CC (8 coherent pixels)

Màu 1, có 1 CC (8 coherent pixels)

Màu 2, có 2 CC (6 coherent pixels và 3 incoherent pixels)

Color	0	1	2
Coherent	8	8	6
Incoherent	0	0	3

$CCV = \{(8,0), (8,0), (6,3)\}$

Image Retrieval: Color Coherence Vector - Owlcation



Code minh họa

```
import cv2
import numpy as np
from collections import defaultdict

def quantize_colors(image, bins=64):
    """Lượng tử hóa ảnh về số màu nhất định."""
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    quantized = (hsv // (256 // bins)).astype(np.uint8)
    return quantized
```



Code minh họa

```
def compute_ccv(image, tau=100, bins=64):  
    """Tính toán Color Coherence Vector (CCV)"""  
    quantized = quantize_colors(image, bins)  
    height, width, _ = quantized.shape  
  
    # Sử dụng từ điển để lưu số lượng pixel  
    region_sizes = defaultdict(int)
```




Code minh họa

```
def compute_ccv(image, tau=100, bins=64):  
    ...  
    # Duyệt qua từng pixel và đếm số lượng màu  
    for y in range(height):  
        for x in range(width):  
            color = tuple(quantized[y, x])  
            region_sizes[color] += 1
```



Code minh họa

```
def compute_ccv(image, tau=100, bins=64):  
    ...  
    # Phân loại coherent và incoherent  
    coherent = defaultdict(int)  
    incoherent = defaultdict(int)  
  
    for color, size in region_sizes.items():  
        if size >= tau:  
            coherent[color] = size  
        else:  
            incoherent[color] = size  
    return coherent, incoherent
```

Color Coherence Vector (CCV)

- **Ưu điểm:** CCV tốt hơn Color Histogram ở chỗ nó có thêm thông tin về độ gắn kết không gian, giúp nhận diện đối tượng chính xác hơn.



<https://www.cs.cornell.edu/~rdz/Papers/PZM-MM96.pdf>

Histogram rank: 88. CCV rank: 38.

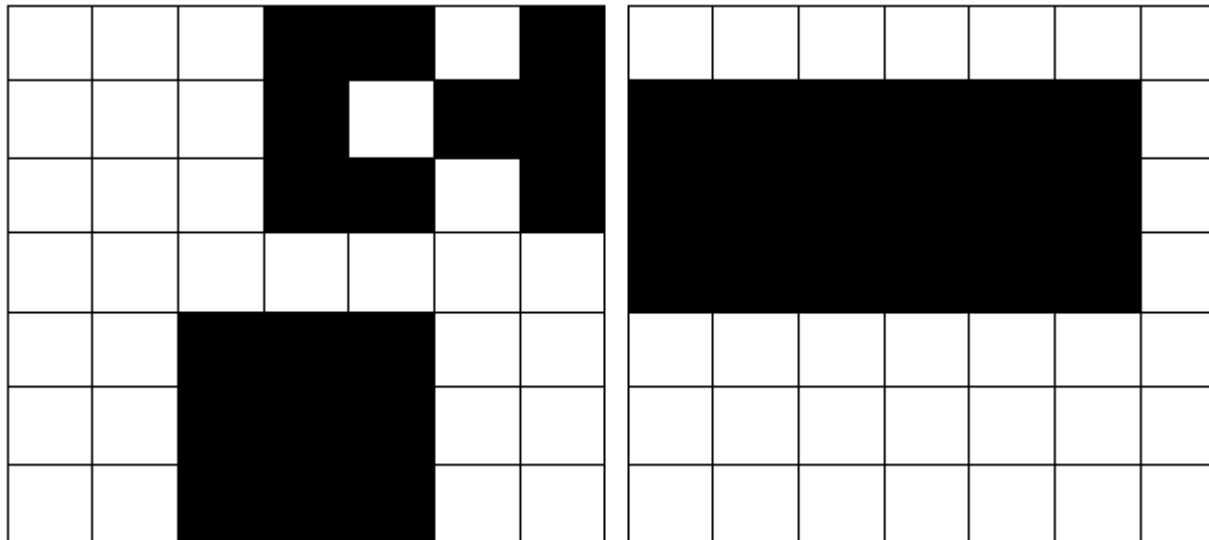


Color Coherence Vector (CCV)

- **Ưu điểm:** CCV tốt hơn Color Histogram ở chỗ nó có thêm thông tin về độ gắn kết không gian, giúp nhận diện đối tượng chính xác hơn.

Color Coherence Vector (CCV)

- Hạn chế:
 - Phụ thuộc vào giá trị ngưỡng τ



Nếu chọn $\tau=8$,
hai ảnh này có CCV
giống nhau

[Image Retrieval: Color Coherence Vector - Owlcation](#)





Tham khảo thêm

- <https://www.pinecone.io/learn/series/image-search/color-histograms/>
- <https://www.youtube.com/watch?app=desktop&v=I3na13AESjw>
- <https://slideplayer.com/slide/4904745/>
- [Image Retrieval: Color Coherence Vector – Owlcation](#)
- [U:PAPERSMM96FINALPC.DVI](#)